



Ejercicios de Machine Learning

Inteligencia Artificial y Mini-Robots (2023251)

Diego Fernando Bustos Horta, dbustosh@unal.edu.co

Julián Mateo Vargas Riveros, jvargasri@unal.edu.co

2026-10-04

Informe de desarrollo — ejercicios de Inteligencia Artificial

Este repositorio reúne la solución de los cinco ejercicios propuestos en la sección 7.11 de IA.md. Cada punto tiene sus fuentes, código y evidencia en una carpeta independiente.

1. Skills para apoyar el trabajo con IA

Se definieron cuatro skills locales, cada una con un archivo SKILL.md que describe cuándo usarla, sus pasos y su resultado esperado:

Skill	Función
capturar-requerimientos	Extraer requisitos y mantener trazabilidad con los documentos fuente.
documentar-ticket	Convertir una necesidad en un ticket con alcance y criterios de aceptación.
implementar-y-probar	Guiar la implementación y su validación.
explicar-trabajo	Establecer la estructura de una explicación HTML del trabajo realizado.

Se aplicaron al documento P1.pdf como ejemplo de captura y organización del trabajo. El resultado está en 01/P1-Skills.md, con los aportes separados por skill. En ese ejemplo no se generó una página HTML ni se implementó software: el archivo documenta el plan y la explicación textual.

2. Chatbot RAG industrial

Se construyó un chatbot para consultar manuales de **pulidora, taladro y compresor**. Hay cuatro PDF en 02/data, incluidos dos documentos de compresor. Las versiones Markdown procesadas están en 02/rag/data.

El flujo toma el texto del manual, lo limpia y divide en fragmentos; luego genera embeddings con **embeddinggemma** mediante Ollama y los guarda en SQLite. Para responder, combina similitud semántica con coincidencia de términos, filtra por herramienta cuando la pregunta la identifica y envía los fragmentos recuperados a **gemma3:1b**. La respuesta incluye el nombre del documento y el número de fragmento.

Código y evidencia:

- Pipeline, consulta y uso: 02/rag/README.md.
- Preguntas y respuestas generadas: 02/preguntas_y_respuestas.md.

Ejemplo de ejecución, desde la raíz del repositorio:

```
python3 02/rag/scripts/build_index.py
python3 02/rag/scripts/embed_with_ollama.py embeddinggemma 2
python3 02/rag/scripts/chat.py "¿Qué debo revisar antes de encender la pulidora?"
```

La calidad de las respuestas aún requiere revisión humana: algunos manuales extraídos contienen texto desordenado o varios idiomas, y la evaluación registra casos con fuentes mezcladas o respuestas incompletas. La herramienta sirve como prototipo académico; las instrucciones de seguridad deben confirmarse en el manual original.

3. Chatbot para los documentos del curso

Se creó un segundo RAG, independiente del industrial, sobre los siete documentos de `03/data/mds`: introducción a la IA, autómatas celulares, algoritmos genéticos, programación genética, redes neuronales, Machine Learning e IA generativa.

El sistema indexa los documentos, crea embeddings con `embeddinggemma`, recupera fragmentos relevantes y genera respuestas con `gemma3:1b`, incluyendo las fuentes consultadas. La implementación y las instrucciones están en `03/rag/README.md`.

```
python3 03/rag/scripts/build_index.py
python3 03/rag/scripts/embed.py embeddinggemma 4
python3 03/rag/scripts/chat.py "¿Qué diferencia hay entre algoritmos genéticos y programación genética?"
```

La batería de evaluación contiene **21 preguntas**, tres por documento. El archivo `03/preguntas_y_respuestas.md` se actualiza después de cada respuesta y puede estar parcialmente completado mientras Ollama trabaja. El progreso se consulta con:

```
python3 03/rag/scripts/progreso.py --watch
```

Para reanudar las preguntas pendientes: `python3 03/rag/scripts/evaluate_questions.py`. Las respuestas automáticas deben revisarse antes de presentarlas como contenido académico validado.

4. Agente de mantenimiento

Se implementó un agente de consola independiente de los dos RAG. Usa SQLite para persistir equipos, repuestos, existencias, órdenes de trabajo e historial de fallas. Permite buscar un repuesto por código, consultar su cantidad y ubicación, crear una orden con falla y equipo, y registrar cambios de estado en el historial. Crear una orden también registra la falla; los eventos posteriores se conservan para mantener trazabilidad.

El código y las instrucciones están en `04/agente.py` y `04/README.md`. El comando `demo` carga datos ficticios para reproducir el ejercicio:

```
python3 04/agente.py demo
python3 04/agente.py buscar-repuesto REP-DIS-01
python3 04/agente.py consultar-almacen REP-DIS-01
python3 04/agente.py crear-orden --equipo EQ-PUL-01 --falla "Vibración excesiva" --repuesto REP-DIS-01
python3 04/agente.py actualizar-historial --orden 1 --estado en_proceso --nota "Disco inspeccionado"
```

Las pruebas de `04/pruebas_y_resultados.md` documentan comandos y respuestas reales. La última ejecución registró **7/7 comprobaciones aprobadas**, incluidas cinco pruebas automáticas. Se pueden repetir con `python3 04/evaluar.py`. En una base que ya contenga órdenes, se debe usar el ID devuelto por `crear-orden` en lugar del 1 del ejemplo.

5. Uso de un LLM con ESP32

En `05.md` se analizó la viabilidad de usar un microcontrolador ESP32 con un LLM. La propuesta es utilizar el ESP32 para capturar datos y comunicarse por Wi-Fi con un servidor o API que ejecute el modelo. Se describen las limitaciones de memoria, procesamiento, almacenamiento, conexión, latencia, consumo, costos y privacidad. Este punto es un análisis conceptual; no incluye firmware ni montaje físico.

6. Requisitos para ejecutar los chatbots

- Python 3.10 o posterior y Ollama instalados localmente.
- Modelos `embeddinggemma` y `gemma3:1b` disponibles en Ollama.
- Servicio de Ollama accesible en `http://localhost:11434`.
- Los índices SQLite se generan con los comandos de cada punto; no forman parte del código fuente versionado.

El agente del punto 4 solo necesita Python 3. Los detalles de uso y regeneración están en los README de cada carpeta.